

Assignment 2 (Weeks 3 and 4)

Part I: Short Answer

Q1

The query fails because NULL represents an unknown value rather than an ordinary value. Comparisons using = with NULL never evaluate to TRUE, so no rows satisfy the condition. The corrected query is:

```
SELECT COUNT(*)  
FROM Student  
WHERE phone_number IS NULL;
```

Aggregate functions such as AVG, SUM, MIN and MAX ignore NULL values because they represent missing information. Including unknown values in calculations would produce unreliable results.

Q2

Yes. The DISTINCT keyword is redundant because GROUP BY department already returns exactly one row for every department. Therefore DISTINCT does not remove any additional rows.

```
SELECT department, COUNT(*)  
FROM Employee  
GROUP BY department;
```

Q3

If a LEFT JOIN returns more rows than an INNER JOIN, then some customers do not have matching records in the Order table. INNER JOIN excludes these customers, while LEFT JOIN keeps them and fills the Order columns with NULL values.

Q4

The query is invalid because employee_name is neither grouped nor aggregated.

```
SELECT department, AVG(salary)  
FROM Employee  
GROUP BY department;
```

If SQL allowed the original query, it would have no logical way to decide which employee name should be displayed for each department because multiple employees belong to the same department.

Q5

WHERE is evaluated before GROUP BY, so AVG(salary) has not been computed when the WHERE clause executes.

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

WHERE filters rows **before** grouping occurs, to filter based on an aggregate, we must use HAVING which filters groups **after** aggregation.

Q6

(a) A non-correlated subquery is appropriate because the overall average price is calculated once.

```
SELECT *
FROM Product
WHERE price >
(SELECT AVG(price) FROM Product);
```

(b) A correlated subquery is appropriate because each employee must be compared with the average salary of that employee's department.

```
SELECT *
FROM Employee e
WHERE salary >
(
SELECT AVG(salary)
FROM Employee
WHERE department=e.department
);
```

A correlated subquery may execute once for every outer row and can therefore be slower on large tables.

Part II

(a)

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Orders o
ON c.customer_id=o.customer_id
WHERE o.order_id IS NULL;
```

A LEFT JOIN preserves all customers and allows those without matching orders to be identified through NULL values.

(b)

```
SELECT restaurant_id,
AVG(order_total) AS average_order_value
FROM(
SELECT o.order_id,o.restaurant_id,
SUM(m.price*oi.quantity) AS order_total
FROM Orders o
JOIN OrderItem oi ON o.order_id=oi.order_id
JOIN MenuItem m ON oi.item_id=m.item_id
GROUP BY o.order_id,o.restaurant_id
)t
GROUP BY restaurant_id
HAVING COUNT(*)>5;
```

The inner query calculates each order's value, while the outer query averages those values. HAVING filters restaurants after grouping.

(c)

```
SELECT restaurant_id,
AVG(order_total) AS avg_order_value
FROM(
SELECT o.order_id,o.restaurant_id,
SUM(m.price*oi.quantity) AS order_total
FROM Orders o
JOIN OrderItem oi ON o.order_id=oi.order_id
JOIN MenuItem m ON oi.item_id=m.item_id
GROUP BY o.order_id,o.restaurant_id
)t
GROUP BY restaurant_id
HAVING AVG(order_total)>
(
SELECT AVG(order_total)
FROM(
SELECT o.order_id,
SUM(m.price*oi.quantity) AS order_total
FROM Orders o
JOIN OrderItem oi ON o.order_id=oi.order_id
JOIN MenuItem m ON oi.item_id=m.item_id
GROUP BY o.order_id
)x
);
```

A non-correlated subquery is sufficient because the platform-wide average is calculated only once.

(d)

```
SELECT restaurant_id,item_name,total_quantity
```

```

FROM(
SELECT m.restaurant_id,m.item_name,
SUM(oi.quantity) AS total_quantity,
RANK() OVER(
PARTITION BY m.restaurant_id
ORDER BY SUM(oi.quantity) DESC
) rnk
FROM MenuItem m
JOIN OrderItem oi ON m.item_id=oi.item_id
GROUP BY m.restaurant_id,m.item_name
) t
WHERE rnk=1;

```

Using only GROUP BY with MAX() cannot reliably return the corresponding row when multiple items share the maximum value.

Part III

Q1

Functional dependencies:

- $\text{student_id} \rightarrow \text{student_name}$
- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$
- $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$
- $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$

Q2

The table is in First Normal Form because every attribute stores atomic values and there are no repeating groups.

Q3

The partial dependencies are:

- $\text{student_id} \rightarrow \text{student_name}$
- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$

These violate 2NF because non-key attributes depend on only part of the composite primary key. For example, changing a student's name requires updating every enrollment record for that student.

Q4

The transitive dependency is:

$$\text{course_id} \rightarrow \text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$$

If an instructor changes office, multiple rows must be updated, creating update anomalies.

Q5

- Student(student_id, student_name)
- Instructor(instructor_id, instructor_name, instructor_office)
- Course(course_id, course_name, instructor_id)
- Enrollment(student_id, course_id, grade, enrollment_date)

Each table isolates a functional dependency and removes redundancy while preserving data integrity.

Part IV

For Part II(c), I chose a non-correlated subquery because the platform-wide average order value needs to be calculated only once. This makes the query easier to read and usually more efficient than recalculating the average for every restaurant. Although a correlated subquery could also solve the problem, it would perform unnecessary repeated computations on larger datasets.

Normalizing the Enrollment table into 3NF reduces redundancy and prevents update, insertion, and deletion anomalies. However, retrieving complete information about a student's enrollment now requires joining several tables such as Student, Course, Instructor, and Enrollment. These joins increase query complexity and may slightly reduce performance. In practice, some systems intentionally keep a limited amount of denormalized data to speed up reports or dashboards where fast read performance is more important than eliminating every redundancy.